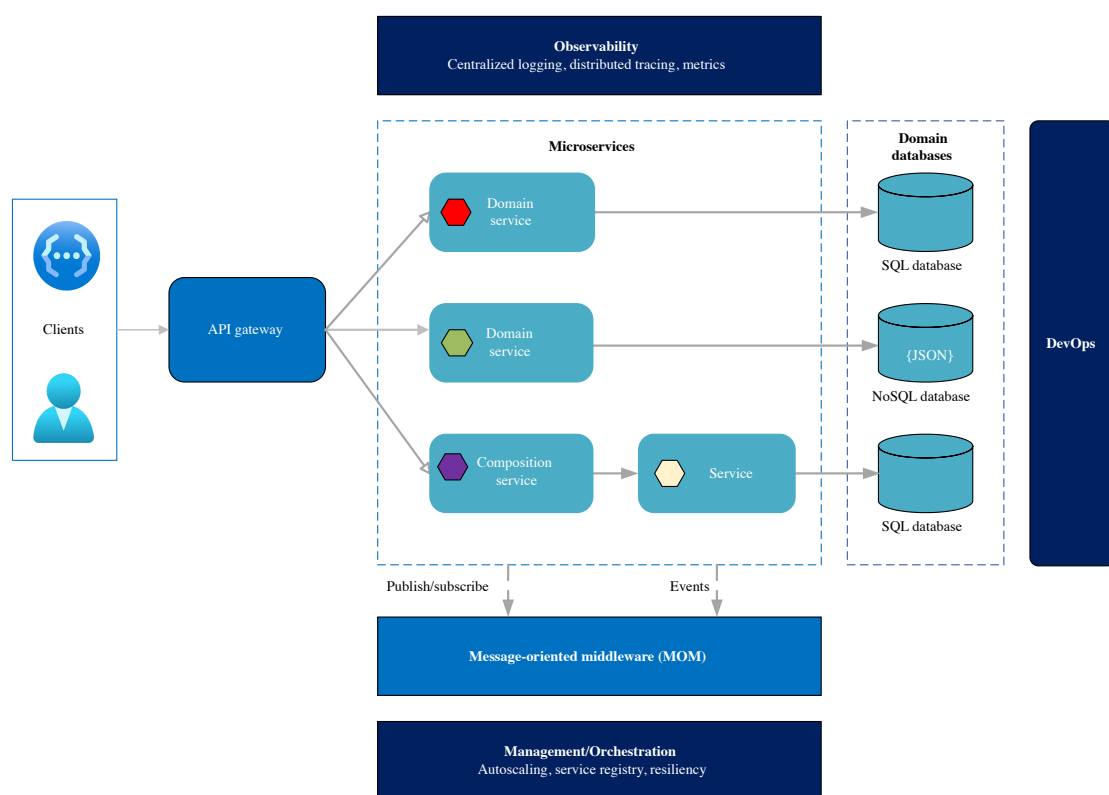


Microservices architecture style

Microservices are a popular architectural style for building applications that are resilient, highly scalable, independently deployable, and able to evolve quickly. Building a successful microservices architecture requires a fundamental shift in mindset. It goes beyond decomposing an application into smaller services. You must also rethink how systems are designed, deployed, and operated.

A microservices architecture consists of a collection of small, autonomous services. Each service is self-contained and should implement a single business capability within a bounded context. A bounded context is a natural division within a business and provides an explicit boundary within which a domain model exists.



What are microservices?

Microservices are small, independent, and loosely coupled components that a single small team of developers can write and maintain. Each service is managed as a separate codebase, which allows a small team to handle it efficiently. Because services can be deployed independently, teams can update existing services without rebuilding or redeploying the entire application. Unlike traditional models that have a centralized data layer, microservices are

responsible for persisting their own data or external state. They communicate through well-defined APIs, which keeps internal implementations hidden from other services. This architecture also supports polyglot programming, which means that services don't need to share the same technology stack, libraries, or frameworks.

Components

In addition to the services themselves, other components appear in a typical microservices architecture:

- **Management or orchestration:** This management component handles microservices orchestration. It schedules and deploys services across nodes, detects failures, recovers from failures, and enables autoscaling based on demand. A container orchestration platform like Kubernetes typically provides this functionality. In cloud-native environments, solutions such as Azure Container Apps provide managed orchestration and built-in scaling. These tools reduce deployment complexity and operational overhead.
- **API gateway:** The API gateway serves as the entry point for clients. Clients send requests to the API gateway instead of calling services directly. The gateway forwards those requests to the appropriate back-end services. It also handles cross-cutting concerns such as authentication, logging, and load balancing. In cloud-native microservices architectures, lightweight service proxies like Envoy and Nginx support internal service-to-service communication. This type of internal traffic, known as east-west traffic, enables advanced routing and traffic control.
- **Message-oriented middleware:** Messaging platforms like Apache Kafka and Azure Service Bus enable asynchronous communication in microservices by promoting loose coupling and supporting high scalability. They form the foundation of event-driven architectures. This approach allows services to react to events in real time and communicate through asynchronous messaging.
- **Observability:** An effective observability strategy helps teams maintain system reliability and resolve problems quickly. Centralized logging brings logs together to support easier diagnostics. Real-time monitoring with application performance monitoring agents and frameworks like OpenTelemetry provides visibility into system health and performance. Distributed tracing tracks requests across service boundaries. It helps teams find bottlenecks and improve performance.
- **Data management:** A well-designed database architecture supports autonomy and scalability. Microservices often use polyglot persistence by choosing different database types, such as SQL or NoSQL, based on each service's specific needs. This approach aligns

with domain-driven design (DDD) and the idea of bounded context. Each service owns its data and schema. This ownership reduces cross-service dependencies and allows services to evolve independently. This decentralized model improves flexibility, performance, and system resilience.

Benefits

- **Agility:** Because microservices are deployed independently, it's easier to manage bug fixes and feature releases. You can update a service without redeploying the entire application and roll back an update if something goes wrong. In many traditional applications, if you find a bug in one part of the application, it can block the entire release process. For example, a bug can stall new features if you need to integrate, test, and publish a bug fix.
- **Small, focused teams:** A microservice should be small enough that a single feature team can build, test, and deploy it. Small team sizes promote greater agility. Large teams tend to be less productive because communication is slower, management overhead increases, and agility diminishes.
- **Small code base:** In a monolithic application, code dependencies often become tangled over time. Adding a new feature might require changes in many parts of the codebase. A microservices architecture avoids this problem by not sharing code or data stores. This approach minimizes dependencies and makes it easier to introduce new features.
- **Mix of technologies:** Teams can pick the technology that best suits their service by using a mix of technology stacks as appropriate.
- **Fault isolation:** If an individual microservice becomes unavailable, it doesn't disrupt the entire application as long as any upstream microservices are designed to handle faults correctly. For example, you can implement the [Circuit Breaker pattern](#), or you can design your solution so that the microservices communicate with each other by using [asynchronous messaging patterns](#).
- **Scalability:** Services can be scaled independently. This approach lets you scale out subsystems that require more resources without scaling out the entire application. Use an orchestrator such as Kubernetes to add a higher density of services onto a single host, which allows for more efficient resource usage.
- **Data isolation:** Updating a schema is simpler in a microservices architecture because only one microservice is affected. In contrast, monolithic applications can complicate schema changes, since multiple components often interact with the same data. This shared access makes any modification potentially risky.

Challenges

The benefits of microservices come with trade-offs. Consider the following challenges before you create a microservices architecture:

- **Complexity:** A microservices application has more moving parts than the equivalent monolithic application. Each service is simpler, but the entire system as a whole is more complex. Make sure that you consider challenges like service discovery, data consistency, transaction management, and interservice communication when you design your application.
- **Development and testing:** Writing a small service that relies on other dependent services requires a different approach than writing a traditional monolithic or layered application. Existing tools aren't always designed to work with service dependencies. Refactoring across service boundaries can be difficult. It's also challenging to test service dependencies, especially when the application is evolving quickly.
- **Lack of governance:** The decentralized approach to building microservices has advantages, but it can also result in problems. You might end up with so many different languages and frameworks that the application becomes hard to maintain. It might be useful to put some project-wide standards in place, without overly restricting teams' flexibility. This method especially applies to cross-cutting functionality such as logging.
- **Network congestion and latency:** The use of many small, granular services can result in more interservice communication. Also, if the chain of service dependencies gets too long (service A calls B, which calls C...), the extra latency can become a problem. You need to design APIs carefully. Avoid overly chatty APIs, think about serialization formats, and look for places to use asynchronous communication patterns like the [Queue-Based Load Leveling pattern](#).
- **Data integrity:** Each microservice is responsible for its own data persistence. As a result, data consistency across multiple services can be a challenge. Different services persist data at different times, using different technology, and with potentially different levels of success. When more than one microservice is involved in persisting new or changed data, it's unlikely that the complete data change could be considered an atomic, consistent, isolated, and durable (ACID) transaction. Instead, the technique is more aligned to Basically Available, Soft State, Eventual Consistency (BASE). Embrace eventual consistency where possible.
- **Management:** A successful microservice architecture requires a mature DevOps culture. Correlated logging across services can be challenging. Typically, logging must correlate multiple service calls for a single user operation.

- **Versioning:** Updates to a service must not break services that depend on it. Multiple services could be updated at any given time, so without careful design, you might have problems with backward or forward compatibility.
- **Skill set:** Microservices are highly distributed systems. Carefully evaluate whether the team has the skills and experience to be successful.

Best practices

- Model services around the business domain. Use DDD to identify bounded contexts and define clear service boundaries. Avoid creating overly granular services, which can increase complexity and reduce performance.
- Decentralize everything. Individual teams are responsible for designing and building services end to end. Avoid sharing code or data schemas.
- Standardize your technology choices by limiting the number of languages and frameworks that you use. Establish platform-wide standards for logging, monitoring, and deployment.
- Data storage should be private to the service that owns the data. Use the best storage for each service and data type.
- Services communicate through well-designed APIs. Avoid leaking implementation details. APIs should model the domain, not the internal implementation of the service.
- Avoid coupling between services. Causes of coupling include shared database schemas and rigid communication protocols.
- Improve security by using mutual Transport Layer Security (mTLS) for service-to-service encryption. Implement role-based access control and use API gateways to enforce policies.
- Offload cross-cutting concerns, such as authentication and Secure Sockets Layer termination, to the gateway. Service meshes and frameworks like Dapr can also help with common cross-cutting concerns like mTLS authentication and resiliency.
- Keep domain knowledge out of the gateway. The gateway should handle and route client requests without any knowledge of the business rules or domain logic. Otherwise, the gateway becomes a dependency and can cause coupling between services.
- Services should have loose coupling and high functional cohesion. Functions that are likely to change together should be packaged and deployed together. If they reside in separate services, those services end up being tightly coupled, because a change in one

service requires updating the other service. Overly chatty communication between two services might be a symptom of tight coupling and low cohesion.

- Use continuous integration and continuous deployment (CI/CD) pipelines to automate testing and deployment. Deploy services independently and monitor rollout health.
- Isolate failures. Use resiliency strategies to prevent failures within a service from cascading. For more information, see [Resiliency patterns](#) and [Design reliable applications](#).
- Use chaos engineering to test the resiliency of your microservice architecture and its dependencies. Evaluate and improve how the system handles partial failures.
- Implement centralized logging, distributed tracing (OpenTelemetry), and metrics collection to ensure observability.

Antipatterns for microservices

When you design and implement microservices, specific pitfalls frequently occur that can undermine the benefits of this architectural style. Recognizing these antipatterns helps teams avoid costly mistakes and build more resilient, maintainable systems. Avoid the following antipatterns:

- Implementing microservices without a deep understanding of the business domain results in poorly aligned service boundaries and undermines the intended benefits.
- Designing events that depend on past or future events violates the principle of atomic and self-contained messaging. This dependency forces consumers to wait and reduces system reliability.
- Using database entities as events exposes internal service details and often fails to convey the correct business intent, which leads to tightly coupled and unclear integrations.
- Avoiding data duplication at all costs is an antipattern. Using patterns like materialized views to maintain local copies improves service autonomy and reduces cross-service dependencies.
- Using generic events forces consumers to interpret and filter messages. This approach adds unnecessary complexity and reduces clarity in event-driven communication.
- Sharing common libraries or dependencies between microservices creates tight coupling, which makes changes risky and widespread and goes against the principle of self-contained services.

- Exposing microservices directly to consumers results in tight coupling, scalability problems, and security risks. Using an API gateway provides a clean, manageable, and secure entry point.
- Keeping configuration values inside microservices tightly couples them to specific environments, which makes deployments harder. However, externalizing configuration promotes flexibility and environment portability.
- Embedding security logic like token validation directly inside microservices complicates their code and maintenance. Alternatively, offloading security to dedicated components keeps services focused and cleaner.
- Failing to abstract common microservices tasks results in repetitive, error-prone code and limits flexibility. Alternatively, using abstraction frameworks like Dapr simplifies development by decoupling business logic from infrastructure concerns.

Build a microservices architecture

The following articles present a structured approach for designing, building, and operating a microservices architecture.

Use domain analysis: To avoid common pitfalls when you design microservices, use domain analysis to define your microservice boundaries. Do the following steps:

1. [Use domain analysis to model microservices.](#)
2. [Use tactical DDD to design microservices.](#)
3. [Identify microservice boundaries.](#)

Design the services: Microservices require a decentralized and agile approach to designing and building applications. For more information, see [Design a microservices architecture](#).

Operate in production: Because microservices architectures are distributed, you must have robust operations for deployment and monitoring.

Related resources

- [Build a CI/CD pipeline for microservices on Kubernetes](#)
- [CI/CD for microservices architectures](#)
- [Microservices architecture on Azure Kubernetes Service \(AKS\)](#)